Building your own offline file-sharing and communication system for the inevitable downfall of the Internet

—

This document has been adapted from the official PirateBox project which is licensed under Attribution-ShareAlike 4.0 International (CC BY-SA 4.0) for more information about this license please see CreativeCommons.org (`https://creativecommons.org/licenses/by-sa/4.0/`).

The official PirateBox source code is licensed under GNU General Public License Version 3.0 for more information about this license please see gnu.org (`https://www.gnu.org/licenses/gpl-3.0.en.html`).

The official PirateBox project does not endorse this training.



Document Assembled by: Rebecca Trapani (@errbufferoverfl)
piratebox@errbufferoverfl.me

CONTENTS

CONTENTS 2

ABOUT THE PIRATEBOX PROJECT 5

ADAPTATIONS MADE FOR THIS PROJECT 6

CONVENTIONS USED IN THIS DOCUMENT 7

HOW TODAY WILL BE RUN 8

- AFTER THE TRAINING 8
- TRAINERS NOTES 9

BEFORE STARTING 10

- How to Install VirtualBox 10
  - Installing on Windows Hosts 10
  - Installing on Mac OS X 11
  - Installing on Linux 12
    - VirtualBox Driver modules 13
    - Installing VirtualBox from a Debian/Ubuntu package 13
    - Installing VirtualBox for another Linux Distribution 14
- Setting up the Virtual Machine 14
- USB 3.0 Support 17
- Debian Credentials 18
- AFTER THE TRAINING 18
  - Required Tools 18
- Changing the SSID of your PirateBox 19
  - SSID Syntax Rules 20
  - Important Note 20

BUILDING THE PIRATEBOX IMAGE 21

- Note 21

License CC BY-SA 4.0

AFTER THE TRAINING 22
Available make Parameters 22
Accessing Image via chroot 23
Supported Platforms by Image 23

PUTTING THE IMAGE ON YOUR SD CARD 25

SETTING UP YOUR PIRATEBOX 30

POST-INSTALLATION 30
Accessing piratebox.lan on mobile devices 31
Change the password of the alarm user 33
Enable Fake-Timeservice 33
AFTER THE TRAINING 34
Adding a Real Time Clock (RTC) 34
Activate Spare Space on the SD Card or USB Device 34
Activate Kareha Image and Discussion Board 36
Activate Timesave Functionality 36
Activate UPnP Media Server 37
Start UPnP Media Server 37
From a Laptop 38
From an Android device 38
From an iPad/iPhone 38
Streaming Tips 39

FINALLY 39

MODIFYING YOUR PIRATEBOX 40
Changing the SSID and IP Address 40
SSID 40
IP Address 40
"Disabling" ShoutBox 41
Secured Directories 41

License CC BY-SA 4.0

Exchanging Upload-Folder 42
Disabling Shoutbox-Reset after reboot 43
Disable client isolation 43
Enabling Wi-Fi Protected Access (WPA) hotspot 43

MODIFYING THE LOOK AND FEEL 44
Landing Page Text 44
Pictures 45
README File 45

AFTER THE TRAINING 46
Setting Up a Mesh Network 46
What is a Mesh Network? 46
Setting up Mesh Networking on the Raspberry Pi 47
Requirements 47
Prerequisites 47
Software Bridge 47
Reconfigure PirateBox 47
IPv6 Setup 49
Mesh Config Script 50
Setup AVAHI Protocol 51
Finalise 51

Appendix A - make completes successfully 52

Appendix B - make fails: recipe for target 'format' failed 56
Solution 56

License CC BY-SA 4.0

ABOUT THE PIRATEBOX PROJECT

The PirateBox creates an offline wireless network designed for anonymous file sharing, chatting, message boarding and media streaming.

It was originally developed by David Darts (**`https://daviddarts.com/`**), the lead developer is Matthias Strubel (**`http://piratebox.aod-rpg.de/dokuwiki/doku.php`**). Aaron Williamson from the Software Freedom Law Center provided advice on the project and Christiane Ruetten originally ported PirateBox to OpenWrt.

When users join the open PirateBox wireless network and open a browser they are redirected (when captive portal is configured) to the PirateBox home page. From here users can chat anonymously, post images or comment on the bulletin board, watch or listen to media and upload and download files.

No logins are required and no user data is logged. Users remain anonymous – the system is purposely not connected to the Internet in order to subvert tracking and preserve user privacy.

The PirateBox has been used by musicians to share their music at festivals and gigs, by teachers to distribute and collect digital materials from students, by emergency response workers and volunteers to publish local first aid information and community updates. It has also been used by librarians and writers to collect, store, and distribute electronic texts, by conference organizers to distribute conference materials and to provide local wireless commenting during presentations. PirateBox has also been used by coworkers collaborating on projects, and by CryptoParty workshop leaders to securely share cryptographic keys.

License CC BY-SA 4.0

ADAPTATIONS MADE FOR THIS PROJECT

As outlined in CC BY-SA 4.0 all changes made to the original PirateBox project must be indicated in any reasonable manner. Where appropriate all modifications to the source code and tutorial will be listed here.

- Added instructions for changing the SSID and IP address before creating the image
- Added instructions on how to flash custom PirateBox image on to an SD Card
- Changed Message of the Day (MOTD) from "on first boot" instructions to custom MOTD
- Moved "on first boot" instructions into this guide
- Added script-based modifications from piratebox.aod-rpg.de (**`http://piratebox.aod-rpg.de/`**) to this guide

License CC BY-SA 4.0

CONVENTIONS USED IN THIS DOCUMENT

The following convention guide has been adopted and modified from the O'Reilly series.

We use the following font and format conventions for UNIX commands, utilities, and system calls, please note that all script, configuration files, sample interactive sessions or commands, not mentioned in passing will be contained within a solid black border.

- Excerpts from scripts or configuration files are shown in a constant width font:

```
#!/bin/sh
# This script echos "Hello World"
echo Hello World
```

- Sample interactive sessions, showing command line input and corresponding output, will be shown in a constant width font, with user-supplied input in bold:

```
% cat /etc/named.pid
78
```

- If the command must be typed by the superuser (root), we use the sharp, pound, hash, or "hashtag" sign (#) even if run using the **sudo** command:

```
# /etc/named
# sudo /etc/named
```

- Command lines, when they appear exactly as a user would type them, are printed in a constant width font, in bold, when they appear in the body of a paragraph. For example:

  Run **ls** to list the files in a directory.

- Domain names are also printed in constant width font, in bold, when they appear within a paragraph, in the digital document these will hyperlink to their source.

- Filenames are printed in constant width font, in bold; for example:

License CC BY-SA 4.0

The BIND name server boot file is usually in **`/etc/named.boot`**

HOW TODAY WILL BE RUN

Because this training is so hands on, we will be working in a "go at your own pace" style of training. This allows people unfamiliar with *nix, or this sort of training to get the assistance they need without slowing down those who may be more familiar with the technology.

At the end of the document there are a number of appendices included that contain examples for successful builds, as well as examples for common problems we faced during the trail stage of this training, we have also included how we solved these problems.

If you encounter any of these issues we encourage you to check out the provided solution first, and if the problem still persists let either of us know.

AFTER THE TRAINING

We have tried to include most of the information you will need to continue your work on developing and modifying your PirateBox under the "**After the Training**" headers, however static documents rarely stay up to date for long, so we have included a number of helpful links to resources.

- Official Website - **`https://piratebox.cc/`**
- Official Github - **`https://github.com/PirateBox-Dev`**
- Official Getting Started Development Guide - **`https://piratebox.cc/development`**
- PirateBox Reddit - **`https://www.reddit.com/r/Piratebox/`**
- Development Builds - **`http://development.piratebox.de/`**
- Development Forum - **`https://forum.piratebox.cc/list.php?3`**
- PirateBox Forum - **`https://forum.piratebox.cc/`**

License CC BY-SA 4.0

- PirateBox IRC Channel (Freenode #piratebox) - **`https://piratebox.cc/irc`**

TRAINERS NOTES

The irony of requiring the Internet to build an offline "Internet" is not lost on me, be glad for when the eventual downfall of the one true Internet does eventuate you will be one of the few prepared. Godspeed.

License CC BY-SA 4.0

BEFORE STARTING

Before starting you will need to install VirtualBox (can be found on the USB stick provided to you) and copy the Debian virtual machine image over to your laptop.

How to Install VirtualBox

These instructions have been taken from the official VirtualBox manual. For further information, please see the full documentation (**`https://www.virtualbox.org/manual/`**).

Installing on Windows Hosts

VirtualBox runs on the following versions of Windows:

- Windows Server 2008 (64-bit)
- Windows Server 2008 R2 (64-bit)
- Windows 7 (32-bit and 64-bit)
- Windows 8 (32-bit and 64-bit)
- Windows 8.1 (32-bit and 64-bit)
- Windows 10 RTM build 10240 (32-bit and 64-bit)
- Windows Server 2012 (64-bit)
- Windows Server 2012 R2 (64-bit)
- Windows Server 2016 (64-bit)

License CC BY-SA 4.0

1. The VirtualBox installation can be started by double-clicking on its executable file (this contains both 32- and 64-bit architectures)
2. This will display the installation welcome dialog and allow you to choose where to install VirtualBox to and which components to install.
3. Depending on your Windows configuration, you may see warnings about "unsigned drivers" or similar. Please select **Continue** on these warnings as otherwise VirtualBox might not function correctly after installation.
4. The installer will create a "VirtualBox" group in the Windows "Start" menu which allows you to launch the application and access its documentation.
5. With standard settings, VirtualBox will be installed for all users on the local system.

Installing on Mac OS X

VirtualBox runs on the following versions of Mac OS X (64-bit):

- 10.10 (Yosemite)
- 10.11 (El Capitan)
- 10.12 (Sierra)
- 10.13 (High Sierra)

Intel hardware is required, please see VirtualBox's list of known limitations (**`https://www.virtualbox.org/manual/ch14.html`**) for more information.

For Mac OS X hosts, VirtualBox ships in a disk image (dmg) file.

1. Double-click on that file to have its contents mounted.

License CC BY-SA 4.0

2. A window will open telling you to double click on the VirtualBox.mpkg installer file displayed in that window.

3. This will start the installer, which will allow you to select where to install VirtualBox to.

After installation, you can find a VirtualBox icon in the "Applications" folder in the Finder.

Installing on Linux

VirtualBox runs on the following versions of Linux hosts (32-bit and 64-bit). Among others, this includes:

- Ubuntu 14.04 LTS, 16.04 LTS, and 17.04
- Debian GNU/Linux 7 ("Wheezy"), 8 ("Jessie") and 9 ("Stretch")
- Oracle Enterprise Linux 5, Oracle Linux 6 and 7
- Redhat Enterprise Linux 5, 6 and 7
- Fedora 25 and 26
- Gentoo Linux
- openSUSE 13.2

VirtualBox advise that it should be possible to use VirtualBox on most systems based on Linux kernel 2.6 or 3.x using either the VirtualBox installer or by doing a manual installation.

However, the formally tested and supported Linux distributions are those for which they offer a dedicated package.

License CC BY-SA 4.0

VirtualBox Driver modules

With Debian and Ubuntu-based distributions, you must install the right version of the linux-headers, usually whichever of **`linux-headers-generic`** , **`linux-headers-amd64`** , **`linux-headers-i686`** or **`linux-headers-i686-pae`** best matches the kernel version name; and if it exists the linux-kbuild package. Basic Ubuntu releases should have the right packages installed by default.

On Debian based systems linux-headers and build essential can be installed using the following

```
# sudo apt-get install linux-headers-$(uname -r|sed 's,[^-]*-
[^-]*-,,') build-essential
```

On Fedora, Redhat, Oracle Linux and many other RPM-based systems, the kernel version sometimes has a code of letters or a word close to the end of the version name, for example "uek" for the Oracle Enterprise kernel or "default" or "desktop" for the standard SUSE kernels. In this case the package name is kernel-uek-devel or equivalent. If there is no such code, it is usually kernel-devel.

On older SUSE and openSUSE Linux, you must install the kernel-source and kernel-syms packages.

Installing VirtualBox from a Debian/Ubuntu package

First, locate the appropriate package for your distribution on the provided USB device. Use **`dpkg`** to install the Debian package:

```
# sudo dpkg -i <package name>.deb
```

The installer will also try to build kernel modules suitable for the current running kernel. If the build process is not successful you will be shown a warning and the package will be left unconfigured. Please have a look at **`/var/log/vbox-install.log`** to find out why the compilation failed.

License CC BY-SA 4.0

Once VirtualBox has been successfully installed and configured, you can start it by selecting "VirtualBox" in your start menu or from the command line.

Installing VirtualBox for another Linux Distribution

For installation instructions on other supported Linux distributions please see the VirtualBox documentation (**`https://www.virtualbox.org/manual/ch02.html#install-linux-host`**).

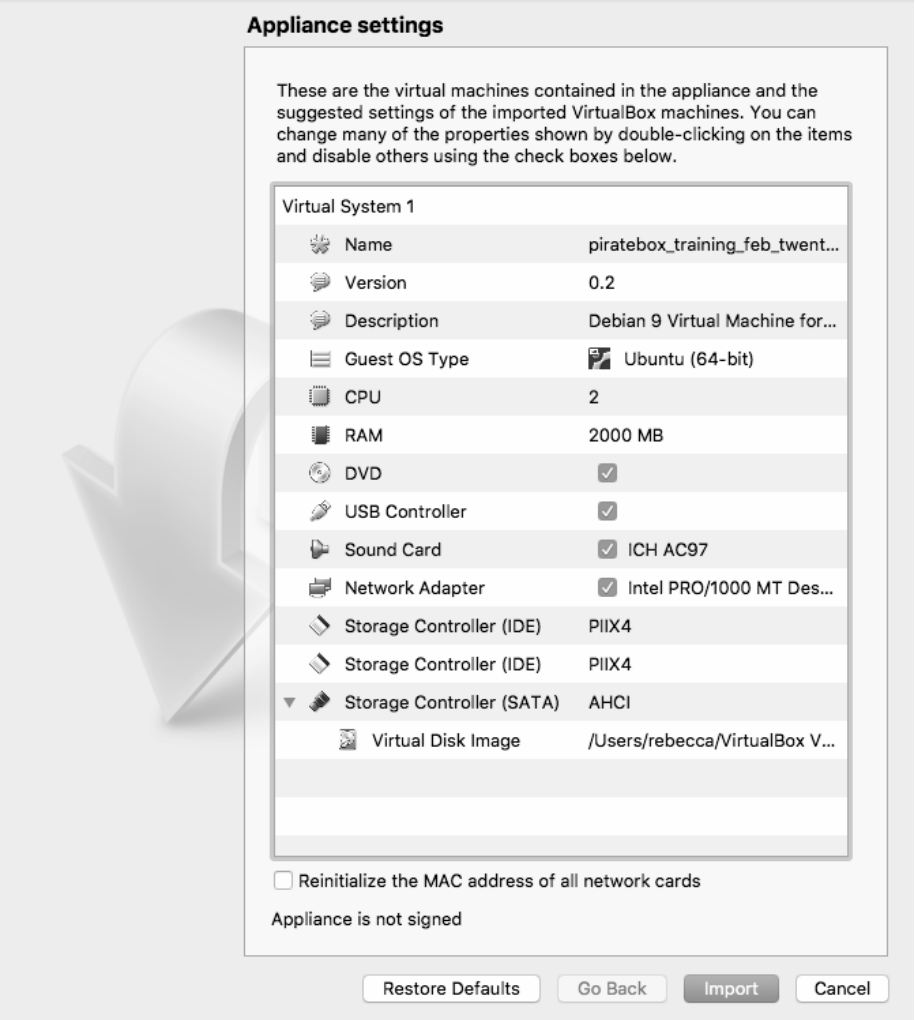
Setting up the Virtual Machine

On your USB device you will find a VirtualBox Open Virtual Appliance file (OVA). To import this into VirtualBox simply copy the file over to your computer, and double-click.

VirtualBox should automatically open and display Appliance Settings for you to configure, as shown in the screen capture on the following page.

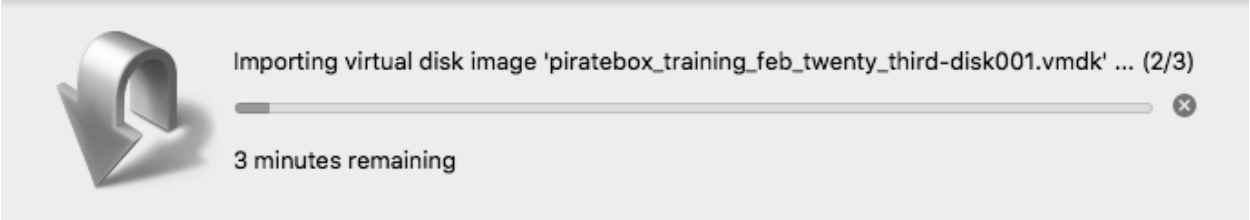
Originally this virtual machine was allocated 2GB of memory and 2 CPUs. This can be increased or decreased depending on your computer's capabilities.

License CC BY-SA 4.0



Click **Import** and VirtualBox will being the process of setting up the virtual machine for you as shown in the screen capture below. Depending on your computer's capabilities this could take upwards of ten minutes.

License CC BY-SA 4.0



Once the virtual machine has been imported it will be listed on the left hand side as powered off, as shown in the screen capture below:



To start the machine, either double click the listing on the left, or click the start button in the toolbar at the top.

Once booted you will be asked to select between **Debian GNU/Linux** or **Advanced Options for Debian GNU/Linux**. Ensure you select **Debian GNU/Linux** by hitting the Enter key.

License CC BY-SA 4.0



Once the machine has booted, you will be able to logging using the credentials provided in the Debian Credentials section. Please note the installation comes pre-installed with Emacs, nano, Sublime, Vim[1].

USB 3.0 Support

While the USB devices provided are not USB 3.0, it is important to note that VirtualBox does not natively support USB 3.0, if you wish to use your own USB 3.0 removable device in the virtual machine you will need to install the platform independent

[1] These are obviously in alphabetical order and not in preference order, otherwise Sublime would be first.

License CC BY-SA 4.0

`Oracle_VM_VirtualBox_Extension_Pack-5.2.6-120293.vbox-extpack` which can be found on the USB device.

You will then be able to enable USB 3.0 support in the virtual machine settings.

Debian Credentials

- **Username:** marvin
- **Password:** password

AFTER THE TRAINING

Required Tools

The Debian environment provided has been pre-configured to save time (so you can skip this step). If you wish to build the PirateBox image at a later time using a different system, you will need to have the following tools installed/configured:

- sudo environment
- losetup from util-linux 2.2 (at least)
- mkfs.vfat
- mkfs.ext4
- wget
- fdisk
- bc
- qemu-system-arm
- qemu-user-static
- pv
- zip

License CC BY-SA 4.0

Depending on the state of your system you may have to install tools such as **make**, **gcc**, or **build-essential**.

On Debian based systems, these can be installed by running:

```
# sudo apt-get install pv qemu-system-arm qemu-user-static zip
```

Packages required may vary from operating system to operating system, non-Debian based users will need to do research into what packages are required.

Changing the SSID of your PirateBox

Because the Raspberry Pi Zero W(ireless) has no Ethernet port, before we setup our PirateBox we will need to change the SSID to make sure you can successfully SSH onto your device instead of someone else's.

Using a file manager, or the command line, open the file : **~/Desktop/arch_rpi_image_prepare/piratebox-ws/piratebox/piratebox/conf/hostapd.conf**

Which should look something like the one below:

```
% cat piratebox-ws/piratebox/piratebox/conf/hostapd.conf
interface=wlan0
driver=nl80211
ssid=change me

channel=1
#auth_algs=1

# Remove # to enable n wifi mode
#ieee80211n=1
#wmm_enabled=1
#ht_capab=[HT40+][SHORT-GI-40][DSSS_CCK-40]
hw_mode=g
```


License CC BY-SA 4.0

```
# Put a # in front of the following line to allow
#  direct client 2 client communication
ap_isolate=1

# Uncomment the following lines to enable WPA
#wpa=2
#wpa_key_mgmt=WPA-PSK
#wpa_pairwise=TKIP CCMP
#rsn_pairwise=CCMP
#wpa_passphrase=Somepassphrase
```

In this file change the SSID variable to anything you wish keeping in mind SSID Syntax Rules.

SSID Syntax Rules

- The maximum length of a Wi-Fi network name is 32 bytes/characters
- SSIDs are case sensitive, thus "abc" is treated as a different name than "aBc"
- Special characters (spaces, periods, dashes, underscores etc) are allowed. It's probably best to avoid the pipe character (vertical line)

Once you have set the SSID to something you are happy with save the file.

Important Note

It is important to note that if you **DO NOT** change the SSID of your PirateBox you run the risk of connecting and SSH-ing into another users PirateBox and not your own.

If you can't think of a unique name, check out one of the following sites that can generate names for you:

- **https://codepen.io/mikedryan/details/vLrgqr**
- **https://mrsharpoblunto.github.io/foswig.js/**
- **http://colinmorris.github.io/rbm/repos/**

License CC BY-SA 4.0

BUILDING THE PIRATEBOX IMAGE

Installed terminals include: **LXTerminal** and **Termit** they are all configured to use Bourne Again SHell (**bash**).

Using a terminal of your choice, navigate to the **/arch_rpi_image_prepare/** directory:

```
% cd Desktop/arch_rpi_image_prepare/
```

Running **make** will acquire all dependencies, install PirateBox and package the image for distribution:

```
% make
```

After the script begins creating the partitions you will be prompted for the user password provided in the Debian Credentials section.

Note

make may take some time to complete, depending on how quickly it can fetch packages and how much CPU and memory has been allocated to the Virtual Machine. If the output freezes or doesn't change for an extended period of time (> 10 minute) give someone a shout.

By default this builds the image for the Raspberry Pi 1 (which includes support for the Raspberry Pi W), for more information on which image to build see the Supported Platforms by Image.

For an example of a successful **make** output, see Appendix A. You will notice a couple of warnings and things like that, as long as **make** does not complete with a error, your image should work fine.

For an example and solution if **make** failed because the **recipe for target 'format' failed**, see Appendix B.

License CC BY-SA 4.0

The versioning scheme of the image is like so:

`${sources}_${arch}_${version}-${buildnum}.img`

Thus you should find an `.img` file named like:

`piratebox_rpi_1.1.1-1-23-01-2018.img`

AFTER THE TRAINING

To make this training easier, we have made hard coded changes to the Makefile, we advise that you download a fresh copy from Github to avoid errors and bugs.

If you wish to build the image for a Raspberry Pi 2 simply pass the ARCH variable:

```
% make ARCH=rpi2
```

Alternatively, if you wish to build an image for each architecture you can run the **`./build_all.sh`** script.

Available `make` Parameters

Different parameters can be passed to the build script, these are described briefly below:

ARCH	May be rpi or rpi2, defaults to rpi
BUILD	Number of the build. Leave empty for a date stamp. Defaults to date stamp
VERSION	Used piratebox version in format x.y.z, defaults to devBuild
SOURCE	May be piratebox or librarybox, defaults to piratebox
BRANCH	The branch of pirtebox-ws used, defaults to master

License CC BY-SA 4.0

Accessing Image via `chroot`

To access the image via **`chroot`** simply mount the image by running:

```
% make mount_image
```

If you need network access within the **`chroot`** you can **`mount proc`** to the mounted image:

```
% mount -t proc proc /mount/root/proc/
```

You can then chroot into the environment:

```
% chroot /mount/root/
```

Don't forget to unmount the image when you are done:

```
% make unmount_image
```

For information on testing the image using QEMU (**`https://github.com/PirateBox-Dev/arch_rpi_image_prepare#testing-via-qemu`**) or dumping the raw image onto an SD card (**`https://github.com/PirateBox-Dev/arch_rpi_image_prepare#dumping-image-to-sd-card`**) see the official Github.

Supported Platforms by Image

There are only two images needed to support all different Raspberry Pi boards.

piratebox-rpi.img

- Raspberry Pi 1 A
- Raspberry Pi 1 A+
- Raspberry Pi 1 B
- Raspberry Pi 1 B+
- Raspberry Pi Zero

License CC BY-SA 4.0

- Raspberry Pi Zero W

piratebox-rpi2.img

- Raspberry Pi 2 B

License CC BY-SA 4.0

PUTTING THE IMAGE ON YOUR SD CARD

For this training we will be using Etcher (`https://etcher.io/`) an open-source, cross-platform, graphical interface application for flashing SD cards.

There are other methods for flashing SD cards, however we recommend using Etcher, if you are more comfortable using these other methods you are more than welcome to, however we have a very limited number of spare SD cards, so if you make a mistake we may not be able to replace your SD card.

Once you have plugged your SD card into the reader, and the reader into the computer, you will need to mount the device onto your Linux virtual machine.

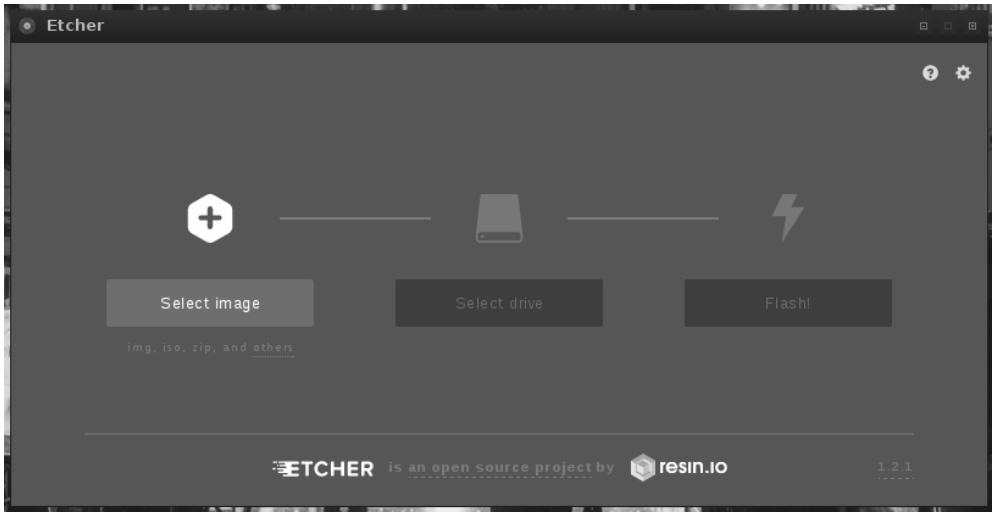
This can be done from two places, in the bottom right hand corner as shown in the screen capture below or under the **Devices** > **USB** menu item. In our example below our SD Card reader is called **Generic USB-2.0-CRW** but this will depend on the device you are using.

License CC BY-SA 4.0

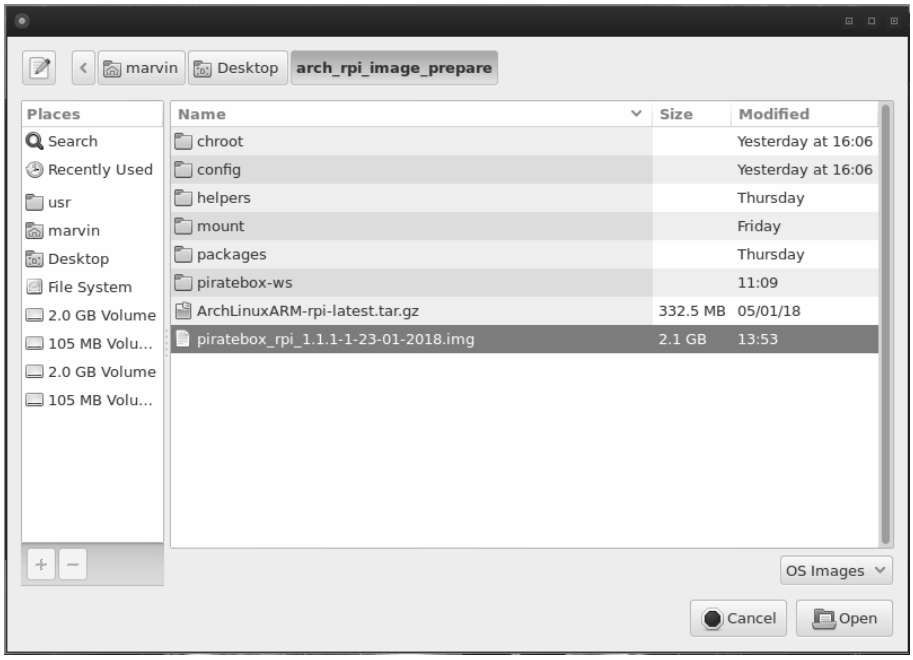


Once the device has been mounted to your virtual machine, start the **Etcher app located on the Desktop**.

License CC BY-SA 4.0

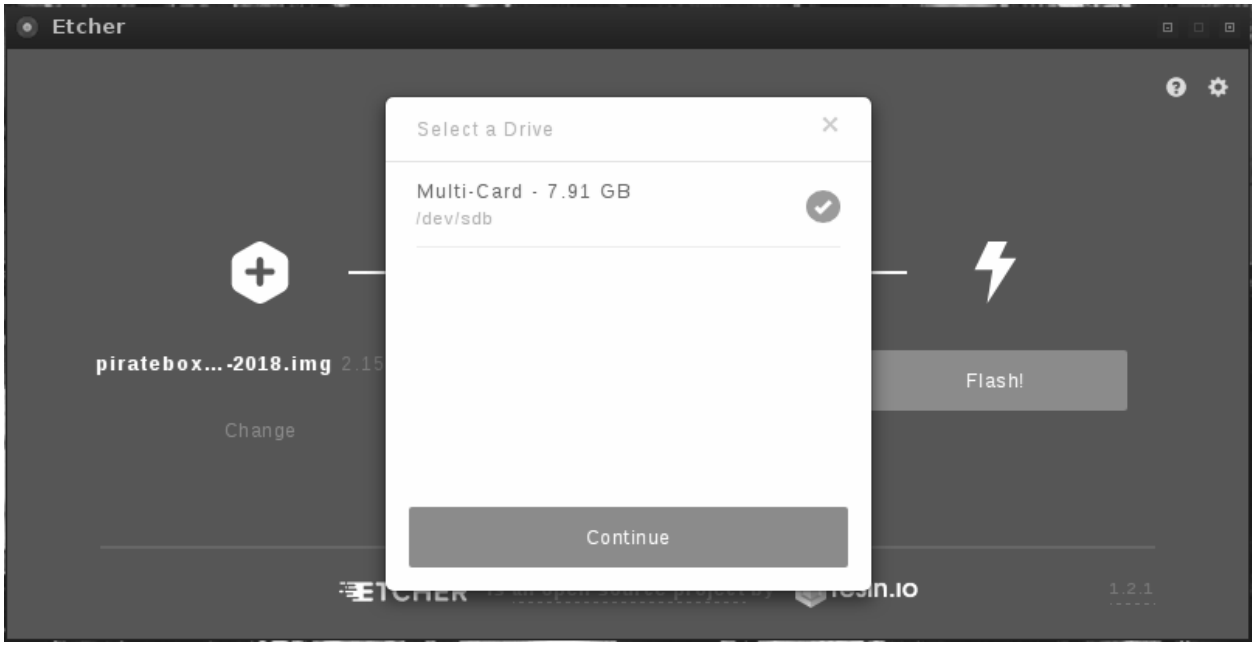


Click **Select Image**, and navigate to location of the image created in the previous step. This should be in **`~/Desktop/arch_rpi_image_prepare/`**.



License CC BY-SA 4.0

Once you have selected an image file, you will be able to select a device, ensure this is the correct device. We aren't liable for any damage to your devices (i.e. data loss due to formatting the wrong device).





You can now your device. You will be prompted to for the user password provided in the Debian Credentials section.

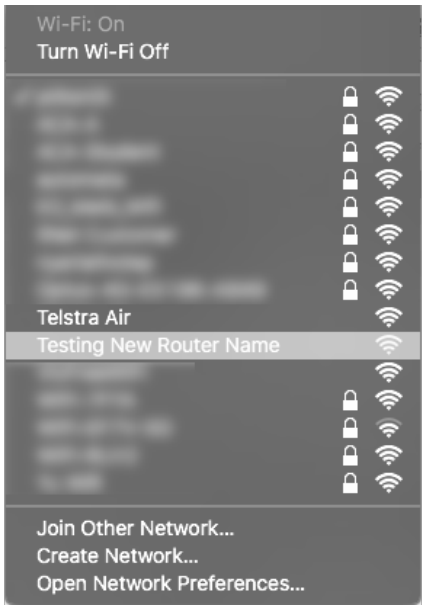
License CC BY-SA 4.0



While you are waiting you can put the case on your Raspberry Pi, or if you've done that you can keep reading.

License CC BY-SA 4.0

SETTING UP YOUR PIRATEBOX



Once the SD card is flashed, and validated, the process is as simple as inserting your SD card into the Raspberry Pi and powering it up. Within a few minutes you should see the device listed in the available Wi-Fi.

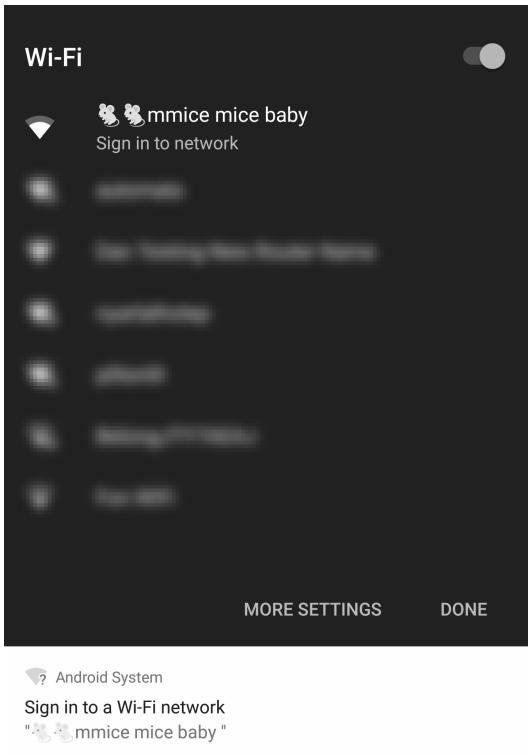
In our example to the right, our PirateBox is called "Testing New Router Name".

If for some reason it's not showing up, let one of us know.

POST-INSTALLATION

In these final steps we will be activating the Kareha Image and Discussion Board and configure and start the UPnP media server.

License CC BY-SA 4.0



Once you have identified your SSID, join it's network, in our example to the left our PirateBox network is named **Mouse Emoji Mouse Emoji mmice mice baby**.

Once you have successfully connected to the network, you can also checkout the bare-bones installation of your PirateBox by visiting **`piratebox.lan`** on either your computer or mobile device.

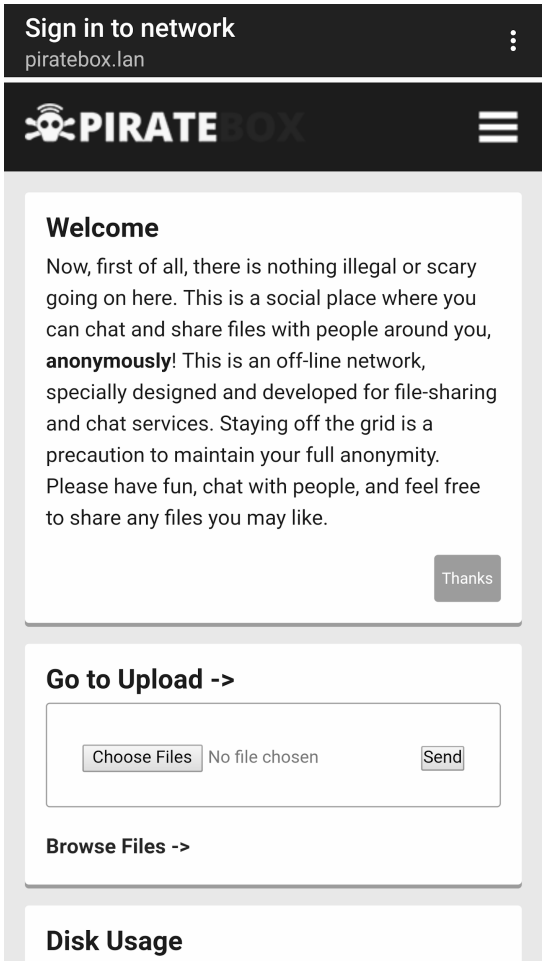
Accessing `piratebox.lan` on mobile devices

In some cases your mobile device may not be able to access **`piratebox.lan`** because they try to be too smart when they notice a Wi-Fi connection does not provide access.

In most cases however, once you connect on your mobile device you will be prompted to "login" to the network, this is not really the case, rather a workaround implemented by the developers so the PirateBox acts as a captive portal, as shown in the screen capture to the left.

License CC BY-SA 4.0

Once you have clicked through to "**Sign in to the Wi-Fi network**" you should be redirected to the **`piratebox.lan`** homepage, as shown on the below left.



Due to the nature of the PirateBox to successfully connect to it, you will need to continue to use that captive portal as trying to use a standard browser will not connect you.

If you have any problems try connecting via your computer first to make sure it's not just the mobile device.

Once you have confirmed your PirateBox is running as expected, using either your virtual machine or host operating system SSH to your PirateBox.

```
% ssh alarm@alarmpi
% ssh alarm@192.168.77.1
# password: alarm
```

The password to the **`alarm`** user is **`alarm`**.

License CC BY-SA 4.0

If you are are using Windows to SSH to the PirateBox, you will need to use a utility such as PuTTY. However, we suggest you connect via the Linux virtual machine provided.

Change the password of the alarm user

First up, you will want to change the password to the alarm user, you should change your password in order to ensure the security of your PirateBox.

```
% ssh alarm@alarmpi
% passwd
```

Enable Fake-Timeservice

Because the PirateBox is completely offline there is no way for it to sync to an Network Time Protocol (NTP) service to keep the time up to date (like your phone or laptop does), therefore we have to set the time and date manually.

```
% ssh alarm@alarmpi
# Set the date and time

# Disable the network time sync
# sudo timedatectl set-ntp false

# Set time to "Fri Feb 23 HH:MM:SS" (Format: CCYYMMDD hhmm)
# Make sure to change HHMM to the current time (24 hour)
# sudo date -s "20180223 HHMM"
% cd /opt/piratebox && sudo ./bin/timesave.sh ./conf/piratebox.conf
install

# Enable on startup
# sudo systemctl enable timesave
```

Because the Pi does not have an onboard Real Time Clock (RTC) like your computer's mainboard when the Pi is turned off and on again, the time displayed will be out by the amount

License CC BY-SA 4.0

of time the device was powered off for. Below, we have included instructions on how to add a Real Time Clock of your own. (RTC not provided in this training).

AFTER THE TRAINING

Adding a Real Time Clock (RTC)

While your Pi is connected to a network it will be able to set its clock correctly using NTP. Without a network connection the system time and date will almost certainly be wrong. This can be solved using an RTC module. This will use a small coin cell battery to keep the time for the Pi even if it is turned off. When the Pi reboots it can set its own internal clock using the time held in by the RTC.

You can use the DS3231 or the DS1307 module, but the one you pick is up to you. We would suggest doing some research before purchasing either module.

Once the module has successfully been installed on your Raspberry Pi you can enable this module of the PirateBox as follows:

```
# sudo systemctl enable rpi_hwclock
```

Activate Spare Space on the SD Card or USB Device

To use the USB device as extended storage you will need to have a USB to Micro USB adapter and format the drive you plan to use as FAT, otherwise you can use the remaining storage of the SD Card (about 4GB of space).

This will allow people to upload content to your device or for you to turn it into a Media Server.

```
% ssh alarm@alarmpi
# Activate spare space on SD Card
# sudo /opt/piratebox/rpi/bin/sdcard_share.sh
```


License CC BY-SA 4.0

```
# Activate spare space on USB Device - before you
# sudo /opt/piratebox/rpi/bin/usb_share.sh
```

Managing a publicly readable, writable service comes with its own risks. It is up to you as the maintainer to take the appropriate steps to ensure that your PirateBox is not abused by other people.

License CC BY-SA 4.0

Activate Kareha Image and Discussion Board

The following commands will enable the Image and Discussion board, an example is shown in the screen capture below:




```
% ssh alarm@alarmpi
# sudo /opt/piratebox/bin/board-autoconf.sh
```

Activate Timesave Functionality

Activating the timesave functionality allows the device to be powered down and on boot retaining the time previously set. It's important to note because it is not using Network Time Protocol (NTP) or a Real Time Clock (RTC) the time will actually be the time the device was powered off rather the time now. This happens because the PirateBox has no way to retain the time while powered off. It is however possible to add an RTC so the PirateBox can retain the time while powered down.

```
% ssh alarm@alarmpi
```


License CC BY-SA 4.0

```
# sudo /opt/piratebox/bin/timesave.sh \
/opt/piratebox/conf/piratebox.conf install
# sudo systemctl enable timesave
```

Activate UPnP Media Server

Your PirateBox includes the ReadyMedia Media Server which automatically streams video and music files to computers and other handheld devices. This means that any video or audio files uploaded to the PirateBox will automatically begin streaming on the PirateBox wireless network.

```
% ssh alarm@alarmpi
# sudo cp /etc/minidlna.conf /etc/minidlna.conf.bkp
# sudo cp /opt/piratebox/src/linux.example.minidlna.conf
/etc/minidlna.conf
```

You can also modify the Media Server configuration file:

```
% ssh alarm@alarmpi
# sudo nano /etc/minidlna.conf
```

Start UPnP Media Server

```
% ssh alarm@alarmpi
# sudo systemctl start minidlna
# sudo systemctl enable minidlna
```

To watch or listen to streaming media files on your device, you will need to use a UPnP compliant media player. Fortunately, there are several good free options available for laptops and other handheld wireless devices. It should be noted that the trainer does not endorse any of the following software, and the reader should do sufficient research to ensure the safety and security of each piece of software before use. Each of the following is suggested on the official PirateBox website.

License CC BY-SA 4.0

From a Laptop

If you plan to connect to PirateBox with a laptop computer, you should first download and install the Kodi Media Player (**`https://kodi.tv/download`**). You can learn more about the Kodi Project here (**`https://kodi.tv/about`**).

Once you have installed Kodi, follow these steps to access the media files hosted on the PirateBox network:

1. Power up your PirateBox and join the PirateBox wireless network
2. Open Kodi on your laptop, and from the main screen click on the **VIDEOS** > **Files** link
3. Click on the **Add Videos...** link and then click the **Browse** button
4. Scroll down to click on the **UpNP Devices** link and then click on the **PirateBox DLNA** link
5. Click the **OK** button and then again on the second **OK** button
6. Now select the **PirateBox DLNA** link and begin browsing the video files on your PirateBox

Note that to listen to music streaming from your PirateBox, you can repeat the steps above but by clicking the **MUSIC** > **Files** link in step #2.

From an Android device

If you wish to connect to your PirateBox with an Android device, you can download and install **BubbleUPnP** and **MX Player** from the Google Play Store.

From an iPad/iPhone

If you wish to connect to PirateBox with an iPhone or iPad, you can first download and install **Ace Player** from the iOS App Store.

To use Ace Player, join the PirateBox's wireless network from your iPhone or iPad and open Ace Player.

License CC BY-SA 4.0

Add the PirateBox DLNA server as a new media server and browse to the files on your device to find a video or audio file.

The media on the PirateBox should begin streaming to your iPhone or iPad within a couple of seconds.

Streaming Tips

To improve your video streaming experience, it is best to optimize the video files by first converting them to an appropriate size and video codec.

FINALLY

Your PirateBox now has all the basic functionality implemented and is now ready to use! From here you can either begin customising the installation, or you're free to enjoy the rest of your afternoon!

Make sure you take your Raspberry Pi, case, MicroSD card, USB device, and power adapter with you!

If you have a power pack similar to that used to charge your phone you will be able to power your PirateBox while at the con!

License CC BY-SA 4.0

MODIFYING YOUR PIRATEBOX

Changing the SSID and IP Address

Once the image is built, you do not need to rebuild the image to change any of the settings, we did this only so there would not be SSID and addressing conflicts.

SSID

To change the SSID to something different open the **/opt/piratebox/conf/hostapd.conf** file on your PirateBox and change the following line, keeping in mind SSID syntax rules.

```
% ssh alarm@alarmpi
% nano /opt/piratebox/conf/hostapd.conf
ssid=piratebox: free share
```

Restart your PirateBox Scripts by running:

```
# sudo systemctl restart piratebox
```

IP Address

To change the IP address your PirateBox is accessible at open the **/opt/piratebox/conf/piraetbox.conf** file on your PirateBox and change the following line.

```
% ssh alarm@alarmpi
% nano /opt/piratebox/conf/piraetbox.conf
# Network
NET=192.168.77
```

If you want to change the IP address of your PirateBox, you can modify the following line in **/opt/piratebox/conf/piraetbox.conf** and replace the 1 with a different value:

```
% ssh alarm@alarmpi
```


License CC BY-SA 4.0

```
% nano /opt/piratebox/conf/piraetbox.conf
# IP-SHORT (is stringed together)
# Which IP is your Box??  i.e. 1 => 192.168.77.1
IP_SHORT=1
```

Restart your PirateBox Scripts by running:

```
# sudo systemctl restart piratebox
```

"Disabling" ShoutBox

ShoutBox is the chat module shown on the home page. If you wish to disable it you will need to open **/opt/piratebox/conf/piratebox.conf** on the PirateBox.

```
% ssh alarm@alarmpi
% nano /opt/piratebox/conf/piratebox.conf

#Chatbox
#  Change the not commented line to en- and disable Chat
# CHAT_BOX_HTML=""
CHAT_BOX_HTML="<iframe height='400px' width='50%' frameBorder='0'
scrolling='0'  src='http://$HOST:8002'></iframe>"
```

Technically, the ShoutBox module is still running, it is just hidden from the view.

Secured Directories

"Secured" directories make it possible to save a folder with a password or upload a file into a directory structure. To enable this feature you will need to open **/opt/piratebox/conf/piratebox.conf** on the PirateBox.

```
% ssh alarm@alarmpi
% nano /opt/piratebox/conf/piratebox.conf

## Droopy Userdir Option?
```


License CC BY-SA 4.0

```
DROOPY_USERDIR=""
# Uncomment the following line to enable Userdirs
#DROOPY_USERDIR="-u True"
```

Remove the `#` in the last line so it looks like the following:

```
## Droopy Userdir Option?
DROOPY_USERDIR=""
# Uncomment the following line to enable Userdirs
DROOPY_USERDIR="-u True"
```

Restart your PirateBox Scripts by running:

```
# sudo systemctl restart piratebox
```

Exchanging Upload-Folder

This will allow you to change where user uploads are saved by changing where the "Browse" link directs users to. To change this feature you will need to open **`/opt/piratebox/conf/piratebox.conf`** on the PirateBox.

NOTE: Avoid special characters in directory-name and ensure the directory exists and has the correct permissions.

```
% ssh alarm@alarmpi
# nano /opt/piratebox/conf/piratebox.conf

#Upload Folder
# Change this if you want to user uploaded data in a subfolder
#   Make sure the folder exists
# UPLOADFOLDER="$FOLDER/Upload"
UPLOADFOLDER=$FOLDER
```


License CC BY-SA 4.0

Disabling Shoutbox-Reset after reboot

By default, the ShoutBox is cleared on startup. To disable this and keep the chat history you will need to open **/opt/piratebox/conf/piratebox.conf** on the PirateBox and change the last line to **"no"**.

```
% ssh alarm@alarmpi
# nano /opt/piratebox/conf/piratebox.conf

# Reset Shoutbox on Startup?
RESET_CHAT="yes"
```

Restart your PirateBox Scripts by running:

```
# sudo systemctl restart piratebox
```

Disable client isolation

By default, it is not permitted to connect directly to another client. To allow clients to connect to one another through your PirateBox you need to disable the "AP Isolation" feature.

```
% ssh alarm@alarmpi
# sudo sed -i -e 's|ap_isolate=1|#ap_isolate=1|g'
/opt/piratebox/conf/hostapd.conf
```

Restart your PirateBox Scripts by running:

```
# sudo systemctl restart piratebox
```

Enabling Wi-Fi Protected Access (WPA) hotspot

If you wish to only allow particular people to access your PirateBox, you can enable a WPA hotspot. It is important to note that WPA is not considered a secure protocol and has a number of known and exploitable vulnerabilities.

License CC BY-SA 4.0

To enable a WPA hotspot change the following configuration file **/opt/piratebox/conf/hostapd.conf**, those options will enable the functionality:

```
% ssh alarm@alarmpi
% nano /opt/piratebox/conf/hostapd.conf
interface=wlan0
ssid=PrivateBox
hw_mode=g
channel=6
ieee80211n=1
wmm_enabled=0
wpa=1
auth_algs=1
wpa_passphrase=verysecurepassword
wpa_key_mgmt=WPA-PSK
wpa_pairwise=TKIP
rsn_pairwise=CCMP
macaddr_acl=0
```

MODIFYING THE LOOK AND FEEL

You can also change the look and feel of your PirateBox and a number of people have mods available on GitHub.

- https://github.com/woodjl1650/Piratebox-Custom-HTML-w-Theater
- https://github.com/thedod/SteamBox

Landing Page Text

The landing page message can be changed in **/opt/piratebox/conf/piratebox.conf**

```
% ssh alarm@alarmpi
# nano /opt/piratebox/conf/piratebox.conf

#Text on droopy
TEXT="<b>1.</b> Learn more about the project <a
```


License CC BY-SA 4.0

```
href=\"http://$HOST:8001/.READ.ME.htm\"
target=\"_parent\"><b>here</b></a>.<p><b>2.</b> Click button below to
begin sharing.</p><b>3.</b> Browse and download files <a
href=\"http://$HOST:8001\" target=\"_parent\"><b>here</b></a>.<br>"
```

Pictures

Pictures on the landing-page are located in the folder **/opt/piratebox/src**. You can simply exchange the files or add new files. If you do create any new files, ensure you change the config to point to these new files in **/opt/piratebox/conf/piratebox.conf**

```
% ssh alarm@alarmpi
# nano /opt/piratebox/conf/piratebox.conf

#Image-SRC
IMAGENAME="piratebox-logo-small.png"
IMAGE="/opt/piratebox/src/$IMAGENAME"
```

README File

The README linked on homepage is located in **/opt/piratebox/src** and will be copied over to your share folder. You need to remove or change them after install.

License CC BY-SA 4.0

AFTER THE TRAINING

The following sections are not necessary to get the most out of your PirateBox, however have been included for the tinkerers, builders and breakers.

Setting Up a Mesh Network

The mesh networking functionality is still currently being tested by the PirateBox developers. While it's fine for small scale networks it is untested for larger production networks.

What is a Mesh Network?

Wikipedia (**https://en.wikipedia.org/wiki/Mesh_networking**) defines Mesh networking (topology) as a type of networking where each node must not only capture and disseminate its own data, but also serve as a relay for other nodes, that is, it must collaborate to propagate the data in the network.

You can read more on Wikipedia (**https://en.wikipedia.org/wiki/Mesh_networking**), or on How Stuff Works (**https://computer.howstuffworks.com/how-wireless-mesh-networks-work.htm**).

The PirateBox uses the B.A.T.M.A.N protocol (**https://en.wikipedia.org/wiki/B.A.T.M.A.N**) which works on Layer 2 and enables every kind of higher protocol such as TCP or UDP.

License CC BY-SA 4.0

Setting up Mesh Networking on the Raspberry Pi

Requirements

- A second Wi-Fi card for Mesh only interface
- The Internet

Prerequisites

First you will need to check the Internet is working and `sudo` is active:

```
% sudo touch /tmp/sudo_test
% sudo rm /etc/resolv.conf
% sudo sh -c 'echo "nameserver 8.8.8.8" > /etc/resolv.conf'
% dig google.de | grep -q -e 192.168.77.1 && echo "error, Internet is
not working!"
```

Software Bridge

Create the network configuration file for the bridge:

```
% cat <<EOF > piratebox_bridge
Description="General bridge interface for PirateBox"
Connection=bridge
Interface=br-lan
BindsToInterfaces=()
ForceConnect=yes
#Enabling DHCP stuff
SkipForwardingDelay=yes
IP=static
Address=(<<IP ADDRESS OF DEVICE>>/24)
EOF
# sudo mv piratebox_bridge /etc/netctl/
# sudo netctl enable piratebox_bridge
# sudo netctl start piratebox_bridge
```

Reconfigure PirateBox

Create the network configuration file for the bridge:

License CC BY-SA 4.0

```
# sudo sed -i -e 's|DO_IFCONFIG="yes"|DO_IFCONFIG="no"|' \
    -e 's|DO_BRIDGE="no"|DO_BRIDGE="yes"|' \
    -e 's|DNSMASQ_INTERFACE="wlan0"|DNSMASQ_INTERFACE="br-lan"|' \
    /opt/piratebox/conf/piratebox.conf
```

Edit the **piratebox.service** file system for system dependencies:

```
# sudo nano /etc/systemd/system/piratebox.service
```

The top of the file should look like this:

```
[Unit]
Description=PirateBox Service
PartOf=netctl@piratebox_bridge.service
```


License CC BY-SA 4.0

IPv6 Setup

```
# sudo sed -i -e 's| radvd|/usr/bin/radvd|g'
/opt/piratebox/init.d/piratebox_alt
```

The run the following commands:

```
% PREFIX="fdc0:ffea"
% FIXED=$(hexdump -n8 -e '/2 ":%03x" ' /dev/random )
% V6_GEN_RESULT="$PREFIX:$FIXED"
% prefix=`</dev/urandom tr -dc a-zA-Z1-9 | head -c8`
% work_hostname="$prefix.piratebox.lan"
# sudo touch /opt/piratebox/conf/node.conf
# sudo sed -i -e \
"s|NODE_CONFIG_ACTIVE='no'|NODE_CONFIG_ACTIVE='yes'|" \
/opt/piratebox/conf/node.conf
# sudo sed -i -e "s|NODE_IPV6_IP=''|NODE_IPV6_IP='$V6_GEN_RESULT'|" \
    -e "s|NODE_NAME=''|NODE_NAME='$prefix'|" \
    /opt/piratebox/conf/node.conf
# sudo sed -i -e 's|IPV6_ENABLE="no"|IPV6_ENABLE="yes"|' \
           -e "s|fdc0:ffea::1|$V6_GEN_RESULT|" \
           /opt/piratebox/conf/ipv6.conf
% echo "IP6=static
Address6=($V6_GEN_RESULT/64)" > ipv6.netctl
sudo sh -c 'cat ipv6.netctl >> /etc/netctl/piratebox_bridge'
% echo "
OK.
Please note the following infos:
   Generated node name: $work_hostname
   Generated IPv6: $V6_GEN_RESULT
   Prefix: /64
"
```


License CC BY-SA 4.0

Mesh Config Script

This step assumes, that you have a working network connection and the Wi-Fi adapter for the Mesh is the 2nd device (phy1).

```
% sudo sh -c 'echo "br_netfilter" > \ /etc/modules-
load.d/netfilter.conf'
# sudo modprobe br_netfilter
# sudo pacman -Sy --noconfirm batctl
% wget \ https://github.com/PirateBox-Dev/PirateBox-
Mesh/releases/download/1.1.4/piratebox-mesh_1.1.4_all_NON_WRT.sh
% chmod a+x piratebox-mesh_1.1.4_all_NON_WRT.sh
# sudo mv piratebox-mesh_1.1.4_all_NON_WRT.sh /bin/piratebox-mesh.sh
# sudo sed -i -e 's|phy0|phy1|g'  \
   -e 's|MESH_CHANNEL=0|MESH_CHANNEL=6|' \
   -e 's|EXCHANGE_MAC="yes"|EXCHANGE_MAC="no"|' \
   /bin/piratebox-mesh.sh
```

Create a start/stop script:

```
% cat <<EOF > mesh.service
[Unit]
Description=PirateBox Mesh Service
PartOf=netctl@piratebox_bridge.service

[Service]
Type=idle
RemainAfterExit=yes
ExecStart=/bin/piratebox-mesh.sh start
ExecStop=/bin/piratebox-mesh.sh stop

[Install]
WantedBy=multi-user.target
EOF
# sudo mv mesh.service /etc/systemd/system
# sudo systemctl daemon-reload
# sudo systemctl start mesh.service
# sudo systemctl enable mesh.service
```


License CC BY-SA 4.0

Setup AVAHI Protocol

```
# sudo mv /etc/avahi /etc/avahi.old
# sudo ln -s /opt/piratebox/conf/avahi/ /etc/avahi
# sudo systemctl start avahi-daemon
# sudo systemctl enable  avahi-daemon
```

Finalise

Once everything has been configured you can reboot:

```
# sudo reboot
```


License CC BY-SA 4.0

Appendix A - **make** completes successfully

```
marvin@starshipheartofgold:~/Desktop/arch_rpi_image_prepare$ make
## Creating rpi image file
* Filename ./piratebox_rpi_1.1.1-1-23-01-2018.img
* Blocksize     512
* Sectors  4194304
* Total size    2147483648 Bytes (2GB)
   2GiB 0:00:16 [ 123MiB/s] [================================>] 100%

## Creating partitions...
cat ./config/fdisk_cmd.txt | sudo fdisk ./"piratebox"_rpi_"1.1.1-1"-
23-01-2018.img > /dev/null

## Formatting partitions...
sudo  mkfs.vfat "/dev/loop1p1" > /dev/null
sudo  mkfs.ext4 "/dev/loop1p2" > /dev/null
mke2fs 1.42.12 (29-Aug-2014)

## Mounting image...
sudo mount "/dev/loop1p1" ./mount/boot
sudo mount "/dev/loop1p2" ./mount/root

# Building piratebox package...
cd piratebox-ws && make
make[1]: Entering directory
'/home/marvin/Desktop/arch_rpi_image_prepare/piratebox-ws'
echo "piratebox-ws_1.1.3" >  piratebox/piratebox/version
echo `git status -sb --porcelain` >> piratebox/piratebox/version
echo ` git log -1 --oneline` >>  piratebox/piratebox/version
tar czf piratebox-ws_1.1.3.tar.gz piratebox
make[1]: Leaving directory
'/home/marvin/Desktop/arch_rpi_image_prepare/piratebox-ws'

## Moving files to their place...
sudo mkdir -p ./mount/root/prebuild > /dev/null
sudo mkdir -p ./mount/root/root/chroot > /dev/null
sudo tar -xf ArchLinuxARM-rpi-latest.tar.gz -C ./mount/root --
warning=none
sudo cp -rv "./packages"/rpi/* ./mount/root/prebuild > /dev/null
sudo cp piratebox-ws/*.tar.gz "./mount/root/root"
```


License CC BY-SA 4.0

```
sudo cp -rv ./chroot/* ./mount/root/root/chroot > /dev/null
sudo mv ./mount/root/boot/* ./mount/boot > /dev/null
sudo cp chroot/wpa_supplicant.conf ./mount/boot/ > /dev/null
sudo cp /usr/bin/qemu-arm-static ./mount/root/usr/bin > /dev/null
sudo sh -c 'echo "/dev/mmcblk0p1 /boot vfat defaults,nofail 0
0" > ./mount/root/etc/fstab'

## chroot'ing to RPi environment...
sudo mv -
f ./mount/root/etc/resolv.conf ./mount/root/etc/resolv.conf.bak >
/dev/null
sudo cp /etc/resolv.conf ./mount/root/etc/resolv.conf > /dev/null
sudo mount -t proc proc ./mount/root/proc/ > /dev/null
sudo mount -o bind /dev ./mount/root/dev/ > /dev/null

# Installing packages...
sudo chroot ./mount/root sh -c "/root/chroot/install_packages.sh >
/dev/null"
warning: file-5.32-1 is up to date -- skipping
warning: findutils-4.6.0-2 is up to date -- skipping
warning: gawk-4.2.0-2 is up to date -- skipping
warning: gettext-0.19.8.1-2 is up to date -- skipping
warning: grep-3.1-1 is up to date -- skipping
warning: groff-1.22.3-7 is up to date -- skipping
warning: gzip-1.8-2 is up to date -- skipping
warning: pacman-5.0.2-2.1 is up to date -- skipping
warning: sed-4.4-1 is up to date -- skipping
warning: texinfo-6.5-1 is up to date -- skipping
warning: util-linux-2.31.1-1 is up to date -- skipping
warning: which-2.21-2 is up to date -- skipping
warning: bash-4.4.012-2 is up to date -- skipping
warning: wireless_tools-30.pre9-1 is up to date -- skipping
warning: perl-5.26.1-1 is up to date -- skipping
warning: iptables-1.6.1-2 is up to date -- skipping
warning: net-tools-1.60.20160710git-1 is up to date -- skipping
warning: wpa_supplicant-1:2.6-11 is up to date -- skipping
warning: dependency cycle detected:
warning: harfbuzz will be installed before its freetype2 dependency
warning: dependency cycle detected:
warning: mesa will be installed before its libglvnd dependency
```


License CC BY-SA 4.0

```
# Configuring sudo...
sudo chroot ./mount/root sh -c "/root/chroot/configure_sudo.sh >
/dev/null"

# Installing PirateBox...
sudo chroot ./mount/root sh -c "/root/chroot/install_piratebox.sh >
/dev/null"
qemu: Unsupported syscall: 384
qemu: Unsupported syscall: 384
--2018-01-23 00:17:00--
http://wakaba.c3.cx/releases/kareha_3.1.4.zip
Resolving wakaba.c3.cx... 104.28.16.86, 104.28.17.86,
2400:cb00:2048:1::681c:1056, ...
Connecting to wakaba.c3.cx|104.28.16.86|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: http://wakaba.c3.cx/releases/Kareha/kareha_3.1.4.zip
[following]
--2018-01-23 00:17:01--
http://wakaba.c3.cx/releases/Kareha/kareha_3.1.4.zip
Reusing existing connection to wakaba.c3.cx:80.
HTTP request sent, awaiting response... 200 OK
Length: 222185 (217K) [application/zip]
Saving to: 'kareha_3.1.4.zip'

kareha_3.1.4.zip    100%[===================>] 216.98K   423KB/s
in 0.5s

2018-01-23 00:17:02 (423 KB/s) - 'kareha_3.1.4.zip' saved
[222185/222185]

qemu: Unsupported syscall: 384
qemu: Unsupported syscall: 384
Removed /etc/systemd/system/dbus-org.freedesktop.resolve1.service.
Removed /etc/systemd/system/multi-user.target.wants/systemd-
resolved.service.
qemu: Unsupported syscall: 384

## Cleaning up chroot...
sudo mv ./mount/root/etc/resolv.conf.bak ./mount/root/etc/resolv.conf
sudo umount ./mount/root/proc/ > /dev/null
sudo umount ./mount/root/dev/ > /dev/null
```


License CC BY-SA 4.0

```
## Unmounting image...
sudo umount ./mount/boot
sudo umount ./mount/root
```


License CC BY-SA 4.0

Appendix B - **make** fails: recipe for target 'format' failed

```
marvin@starshipheartofgold:~/Desktop/arch_rpi_image_prepare$ make
## Creating rpi image file
* Filename ./piratebox_rpi_1.1.1-1-23-01-2018.img
* Blocksize     512
* Sectors  4194304
* Total size    2147483648 Bytes (2GB)
   2GiB 0:00:24 [83.9MiB/s]
[=============================================================>] 100%

## Creating partitions...
cat ./config/fdisk_cmd.txt | sudo fdisk ./"piratebox"_rpi_"1.1.1-1"-
23-01-2018.img > /dev/null
[sudo] password for marvin:

## Formatting partitions...
sudo  mkfs.vfat "/dev/loop2p1" > /dev/null
/dev/loop2p1: No such file or directory
Makefile:77: recipe for target 'format' failed
make: *** [format] Error 1
```

Solution

1. Remove any ***.img** files from **/arch_rpi_image_prepare**
2. Remove any ***.tar.gz** files from **/arch_rpi_image_prepare/piratebox-ws**
3. If problem persists, reboot the virtual machine

License CC BY-SA 4.0